

Deep Learning for Solving Dynamic Models

Session 8 — Economics-informed neural networks

Simon Scheidegger

HEC, University of Lausanne

Based on Raissi, Perdikaris & Karniadakis (2019), *Journal of Computational Physics* 378, 686–707

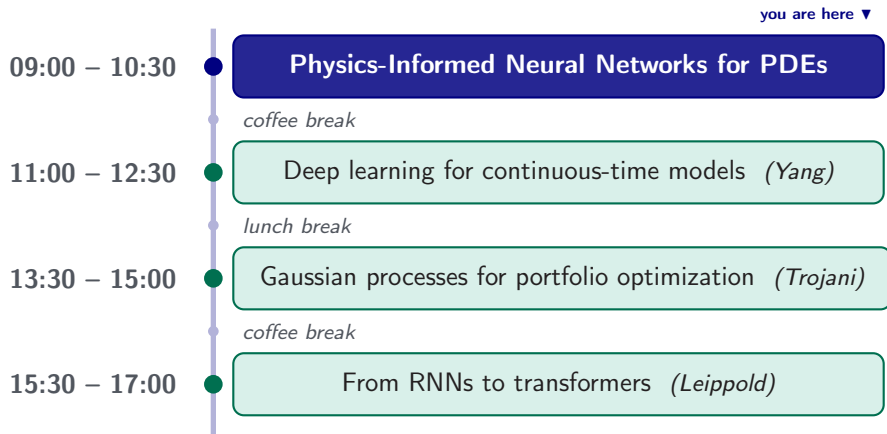
University of Padova

September 24, 2026

Further: Sirignano & Spiliopoulos (2018), Deep Galerkin Method for PDEs in high dimensions.

Al-Aradi, Correia, Naez, Hoshisashi & Schwab (2018), DGM PDE solver.

Day 2 at a Glance



Materials: github.com/sischei/deep_learning_padova_0926 · **Cloud:** nuvolos.cloud

Outline

1. From DEQNs to PINNs

Bridging discrete and continuous time; the PINN loss.

2. PINN Fundamentals

Automatic differentiation for PDEs, collocation, training loop.

3. Soft vs. Hard Boundary Conditions

Trial solutions, 2D Poisson, practical guidance.

4. Economic Applications: HJB Equation

Cake-eating problem, scaled trial solution + hard BCs, Adam $\rightarrow$ L-BFGS, analytical benchmark; DGM as optional high-D architecture.

5. Financial Applications: Black–Scholes

Option pricing via PINNs, comparison with analytical formula.

6. When to Use One, and What Goes Wrong

PINNs vs. grid solvers, inverse problems, failure modes, and which notebooks to run.

Roadmap of This Lecture (90 min)

I. From DEQNs to PINNs (11 min)

- ▶ Discrete vs. continuous time, the PINN loss
- ▶ Finger exercise

II. PINN fundamentals (20 min)

- ▶ Autograd for PDEs, collocation, training loop
- ▶ Adam $\rightarrow$ L-BFGS in FP64; warm-up 1D ODE

III. Boundary conditions (17 min)

- ▶ Soft penalties vs. hard trial solutions
- ▶ 2D Poisson; finger exercise

IV. HJB equations (18 min)

- ▶ Cake-eating, scaled trial solution, hard BCs
- ▶ DGM as an optional high-dimensional architecture

V. Finance: Black–Scholes (8 min)

- ▶ Option pricing against the closed form

VI. When to use one (16 min)

- ▶ PINNs vs. grid solvers; inverse problems
- ▶ Failure modes; what to run

Part I

From DEQNs to PINNs

Recall: The DEQN Loss from Day 1

Yesterday morning (02_DeepEquilibriumNets.pdf) **we trained** a neural network $\mathcal{N}_\theta$ to satisfy economic equilibrium conditions in **discrete time**:

$$\ell_\theta^{\text{DEQN}} = \frac{1}{N} \sum_{i=1}^N \|G(\mathbf{x}_i, \mathcal{N}_\theta(\mathbf{x}_i))\|^2$$

- ▶ $G(\cdot) = 0$: Euler equations, budget constraints, market clearing
- ▶ Training points $\mathbf{x}_i$: simulated from the model's own dynamics
- ▶ **No labeled data needed**, structural equations *are* the loss

Today's question: What if the economic model is formulated in **continuous time**?

Discrete vs. Continuous Time

Discrete Time (DEQN)

Euler equation:

$$u'(c_t) = \beta \mathbb{E}[u'(c_{t+1}) R_{t+1}]$$

State transition:

$$\mathbf{x}_{t+1} = h(\mathbf{x}_t, p(\mathbf{x}_t), \varepsilon_{t+1})$$

Loss: algebraic residual $G(\cdot) = 0$

Continuous Time (PINN)

State evolution:

$$da = (ra - c) dt + \sigma dW$$

HJB equation:

$$\rho V = \max_c \{ u(c) + V_a(ra - c) + \frac{1}{2}\sigma^2 V_{aa} \}$$

Loss: PDE residual $\mathcal{R}(\cdot) = 0$

Key shift: from algebraic residuals to **differential** residuals, we need **derivatives** of the neural network. The diffusion term $\frac{1}{2}\sigma^2 V_{aa}$ is why **second** derivatives matter.

Why Continuous Time Matters

Many frontier models are formulated in continuous time:

- ▶ **Heterogeneous-agent models:** Achdou, Han, Lasry, Lions & Moll (2022), income and wealth distribution dynamics via HJB–KFE systems
- ▶ **Mean-field games:** optimal policy with a continuum of interacting agents
- ▶ **Asset pricing:** Black–Scholes, Heston, jump-diffusion models
- ▶ **Macro-finance:** Brunnermeier & Sannikov (2014), He & Krishnamurthy (2013)

Where this sits in the summer school

Day 1 solved equilibrium conditions in **discrete** time with DEQNs. This morning we do the **continuous-time** counterpart, and at 11:00 Yang takes the same machinery to heterogeneous-agent models, where the HJB is coupled to a Kolmogorov forward equation.

The PINN Loss Function

Core idea (Raissi, Perdikaris & Karniadakis, 2019):

Given a PDE $\mathcal{D}[u](\mathbf{x}) = 0$ on domain Ω with boundary conditions $\mathcal{B}[u](\mathbf{x}) = 0$ on $\partial\Omega$, define:

$$\ell_{\theta}^{\text{PINN}} = \underbrace{\frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{D}[\mathcal{N}_{\theta}](\mathbf{x}_i^r)|^2}_{\text{PDE residual}} + \underbrace{\frac{\lambda}{N_b} \sum_{j=1}^{N_b} |\mathcal{B}[\mathcal{N}_{\theta}](\mathbf{x}_j^b)|^2}_{\text{boundary/initial conditions}}$$

- ▶ $\mathcal{N}_{\theta}$: neural network with parameters θ
- ▶ $\mathbf{x}_i^r$: **collocation points** in the interior of Ω
- ▶ $\mathbf{x}_j^b$: points on the boundary $\partial\Omega$
- ▶ Derivatives $\mathcal{D}[\mathcal{N}_{\theta}]$ computed via **automatic differentiation**

Warm-up ODE: for $u''(x) = -1$ on $(0, 1)$ with $u(0) = u(1) = 0$, use $\mathcal{D}[u] = u'' + 1$ in the interior and $\mathcal{B}[u] = (u(0), u(1))$ on the boundary.

Finger Exercise: Writing a PINN Loss

Exercise (5 min)

Consider the ODE $u''(x) = -1$ on $[0, 1]$ with boundary conditions $u(0) = u(1) = 0$.

1. Write down the PINN residual $\mathcal{R}(x; \theta) = \dots$ using a neural network $\mathcal{N}(x; \theta)$.
2. Write the total PINN loss as a sum of a PDE residual term and a boundary term.
3. What is the exact solution?

Solution: Writing a PINN Loss

Answer

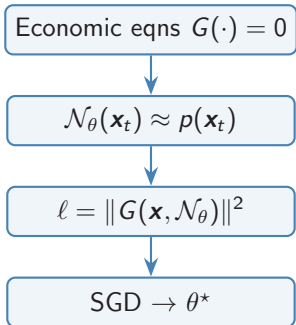
1. $\mathcal{R}(x; \theta) = \frac{\partial^2 \mathcal{N}}{\partial x^2}(x; \theta) + 1.$

2. $\mathcal{L}(\theta) = \underbrace{\frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{R}(x_i; \theta)|^2}_{\text{PDE residual}} + \lambda \underbrace{[\mathcal{N}(0; \theta)^2 + \mathcal{N}(1; \theta)^2]}_{\text{boundary conditions}}$

3. $u(x) = \frac{1}{2}x(1-x)$ satisfies $u'' = -1$, $u(0) = u(1) = 0$. ✓

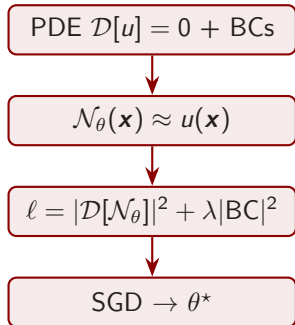
DEQN vs. PINN: Side-by-Side

DEQN (Discrete Time)



← same principle →
+ autograd derivatives

PINN (Continuous Time)



Both approaches are **unsupervised**: the physics (economics) is the loss function. The key addition for PINNs: we differentiate the network via any auto-diff tool (e.g. `torch.autograd.grad`).

Part II

PINN Fundamentals

Automatic Differentiation for PDEs

Key tool: `torch.autograd.grad` with `create_graph=True`.

```
# First derivative dy/dx
dydx = torch.autograd.grad(
    y, x,
    grad_outputs=torch.ones_like(y),
    create_graph=True # keep graph for higher derivs
)[0]
```

- ▶ `create_graph=True`: retains the computational graph so we can differentiate **again**
- ▶ This gives us **algorithmic** derivatives (up to floating-point precision), no finite differences!
- ▶ Works for **any order**: $\frac{\partial^n}{\partial x^n}$ by chaining n calls

Computing Second Derivatives

Chain two autograd calls to get $y''(x)$:

First derivative

```
dydx = torch.autograd.grad(  
    y, x, grad_outputs=torch.ones_like(y),  
    create_graph=True)[0]
```

Second derivative

```
d2ydx2 = torch.autograd.grad(  
    dydx, x, grad_outputs=torch.ones_like(dydx),  
    create_graph=True)[0]
```



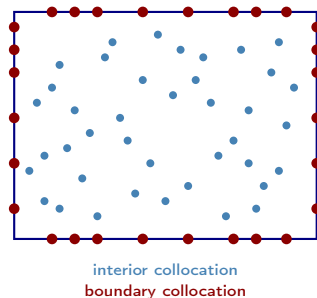
Collocation Points

PINNs evaluate the PDE residual at **randomly sampled** points, no grid required.

1D: interior + boundary



2D: $\Omega = (0, 1)^2$



Points are **resampled** periodically during training to avoid overfitting to a fixed set.

PINN Training Algorithm

Algorithm: Physics-Informed Neural Network Training

Input: PDE $\mathcal{D}[u] = 0$, BCs $\mathcal{B}[u] = 0$, network $\mathcal{N}_\theta$, learning rate η

for epoch = $1, \dots, E$ **do**

 Sample N_r collocation points $\{\mathbf{x}_i^r\}$ in domain Ω

 Sample N_b boundary points $\{\mathbf{x}_j^b\}$ on $\partial\Omega$

 Forward pass: compute $\mathcal{N}_\theta(\mathbf{x}_i^r)$ and $\mathcal{N}_\theta(\mathbf{x}_j^b)$

 Autograd: compute $\mathcal{D}[\mathcal{N}_\theta](\mathbf{x}_i^r)$ via `torch.autograd.grad`

$\ell_\theta = \frac{1}{N_r} \sum_i |\mathcal{D}[\mathcal{N}_\theta](\mathbf{x}_i^r)|^2 + \frac{\lambda}{N_b} \sum_j |\mathcal{B}[\mathcal{N}_\theta](\mathbf{x}_j^b)|^2$

 Update: $\theta \leftarrow \theta - \eta \nabla_\theta \ell_\theta$

end for

Output: Trained $\mathcal{N}_{\theta^*}$ approximating the PDE solution

Note: Unlike DEQNs, training points are **not** simulated from model dynamics, they are drawn uniformly (or adaptively) from the domain.

Optimizer Pipeline: Adam $\rightarrow$ L-BFGS, in FP64

Two stages.

- ▶ **Adam** on resampled collocation: finds a basin.
- ▶ **Deterministic L-BFGS** on a *fixed* grid: closes the residual. Resampling per evaluation breaks the line search.

Why FP64?

- ▶ tanh-MLP second derivatives lose ~ 7 digits in FP32.
- ▶ The Wolfe search compares values at exactly that scale.
- ▶ FP32 plateaus at the noise floor; FP64 keeps going.

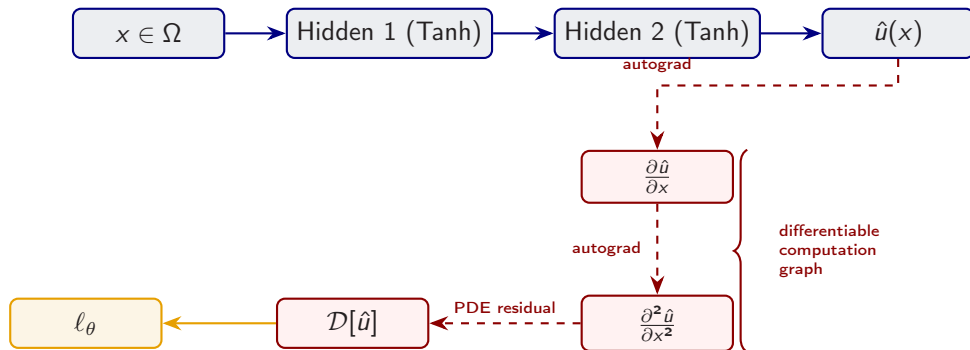
Used in notebooks 04 and 05. Report PDE residuals and solution diagnostics, not the optimizer label.

Practical recipe

1. Cast model + data to float64.
2. Adam, lr 10^{-3} , a few k epochs, resampled collocation.
3. Freeze grid; `torch.optim.LBFGS` (strong-Wolfe).
4. Stop on residual stagnation.

Precision evidence: [arXiv:2505.10949](#).

PINN Architecture



The entire pipeline, forward pass *and* derivative computation, is **differentiable**, enabling end-to-end gradient-based training.

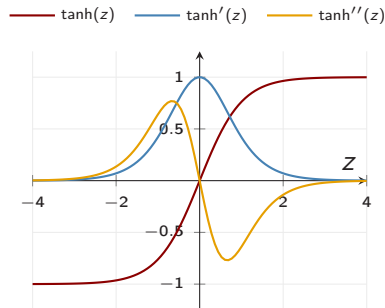
Why Tanh Activations for PINNs?

PDE residuals require smooth derivatives.

- ▶ **ReLU**: C^0 only; its second derivative vanishes almost everywhere, so a u'' residual carries no gradient
- ▶ **Tanh**: C^∞ , all derivatives are well-defined and informative
- ▶ **Softplus**: C^∞ , useful for positivity constraints (e.g. consumption)

Rule of thumb

For PINNs requiring k -th order derivatives, use activations that are at least C^k .



Warm-up: Solving an ODE with a PINN

Problem: $y''(x) = -1$ on $(0, 1)$, $y(0) = y(1) = 0$. **Analytical:** $y(x) = \frac{1}{2}x(1 - x)$.

PINN setup:

1. NN $\mathcal{N}_\theta : [0, 1] \rightarrow \mathbb{R}$, 2 hidden layers of 20 units, tanh
2. Residual: $r(x) = \mathcal{N}_\theta''(x) + 1$
3. Loss: PDE residual + BC penalty (soft, here)

$$\ell = \frac{1}{N} \sum_i r(x_i)^2 + \mathcal{N}_\theta(0)^2 + \mathcal{N}_\theta(1)^2$$

After 5,000 Adam epochs:

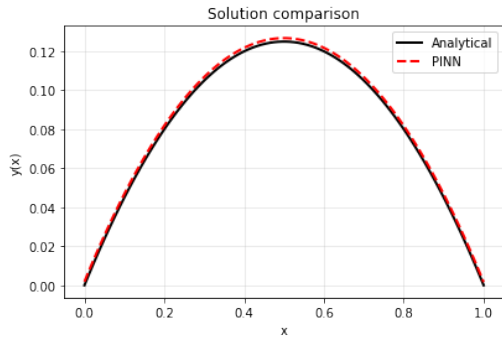
- ▶ Final loss $\approx 2 \times 10^{-5}$, of which the BC term is $\approx 9 \times 10^{-6}$
- ▶ Max absolute error 1.9×10^{-3} , mean 1.8×10^{-3}

Read the error, not the loss

A small loss says the residual is small at the sampled points. Only the comparison against $\frac{1}{2}x(1 - x)$ says the *solution* is right.

Notebook: 01_ODE_PINN_ZeroBCs

Warm-up: PINN vs. Analytical Solution



Left: the network on top of $y(x) = \frac{1}{2}x(1-x)$ — indistinguishable at this scale. Right: the absolute error, which is what actually tells you how well it did. Note the axis: it stays in a narrow band around 1.8×10^{-3} across the domain, with no drift to the boundaries.

Notebook: 01_ODE_PINN_ZeroBCs

Non-Zero Boundary Conditions: The Challenge

Problem: $y''(x) = -1$ on $(0, 1)$ with $y(0) = 1$, $y(1) = 2$.

Analytical solution: $y(x) = -\frac{1}{2}x^2 + \frac{3}{2}x + 1$.

Soft BC enforcement:

$$\ell = \frac{1}{N} \sum_i r(x_i)^2 + \lambda_0 (\mathcal{N}_\theta(0) - 1)^2 + \lambda_1 (\mathcal{N}_\theta(1) - 2)^2$$

Issues with soft enforcement:

- ▶ BCs are only **approximately** satisfied
- ▶ Penalty weight λ requires **tuning**: too small $\Rightarrow$ BC violated; too large $\Rightarrow$ PDE ignored
- ▶ Boundary errors can **propagate** into the interior

$\Rightarrow$ **Motivates hard boundary enforcement** (Part III).

Part III

Soft vs. Hard Boundary Conditions

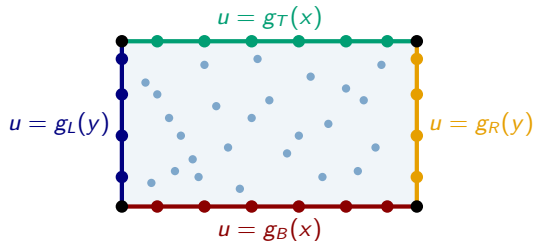
What Are Boundary Conditions?

1D: $y''(x) = -1$ on $(0, 1)$



Interior: enforce $y'' + 1 = 0$
Boundary: enforce $y = \text{prescribed value}$

2D: $\Delta u = f$ on $(0, 1)^2$

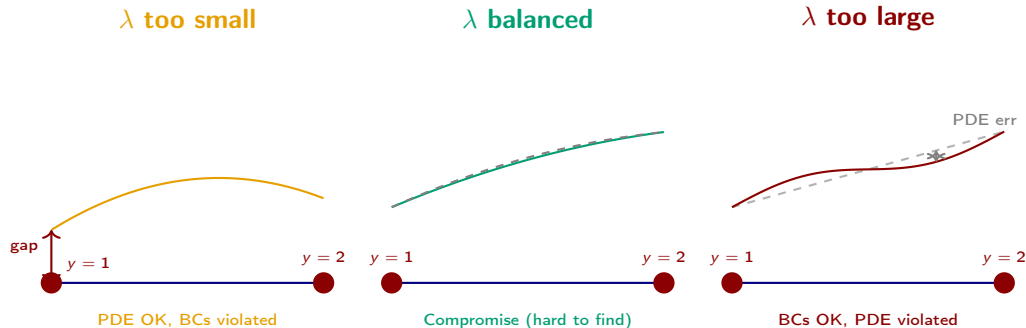


Two enforcement strategies

Soft: add BC violation as a penalty term in the loss.

Hard: build BCs into the network output by construction.

The Problem with Soft Boundary Conditions



Soft BCs require **careful balancing** of the penalty weight λ .
Too small: BCs violated. Too large: PDE ignored. Just right: hard to find *a priori*.

Hard BC Enforcement: Trial Solutions

Idea: construct a **trial solution** that satisfies BCs *by construction*:

$$\hat{y}(x) = A(x) + B(x) \cdot \mathcal{N}_\theta(x)$$

where:

- ▶ $A(x)$: **anchor function** satisfying the BCs exactly, e.g. $A(x) = 1 + x$
- ▶ $B(x)$: **mask function** vanishing at boundaries, e.g. $B(x) = x(1 - x)$
- ▶ $\mathcal{N}_\theta(x)$: free neural network (no BC constraints)

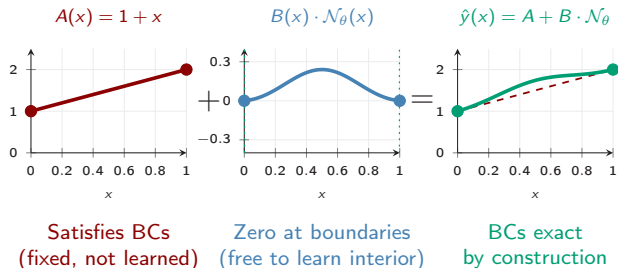
Verification:

$$\hat{y}(0) = A(0) + B(0) \cdot \mathcal{N}_\theta(0) = 1 + 0 = 1 \quad \checkmark$$

$$\hat{y}(1) = A(1) + B(1) \cdot \mathcal{N}_\theta(1) = 2 + 0 = 2 \quad \checkmark$$

$\Rightarrow$ BCs hold **identically**, only the PDE residual loss remains.

Trial Solution: Visual Decomposition



Hard BC: Implementation

```
def A(x): # Anchor: A(0)=1, A(1)=2
    return 1.0 + x
def B(x): # Mask: B(0)=B(1)=0
    return x * (1.0 - x)

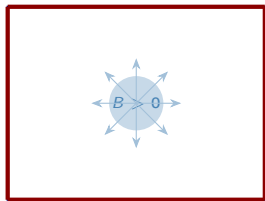
def y_trial(model, x): # Hard-BC trial solution
    return A(x) + B(x) * model(x)

def residual(model, x):
    x.requires_grad_(True)
    y = y_trial(model, x)
    dydx = torch.autograd.grad(y, x, ...)[0]
    d2ydx2 = torch.autograd.grad(dydx, x, ...)[0]
    return d2ydx2 + 1.0 # should be 0
```

Advantage: the loss has **only one term** (PDE residual), no λ to tune!

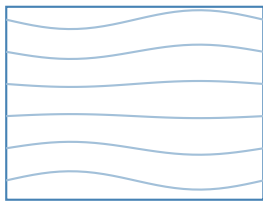
Hard BCs in 2D: Building the Trial Solution

Mask $B(x, y) = x(1-x)y(1-y)$



$B = 0$ on all edges

Network $\mathcal{N}_\theta(x, y)$



Free to take any value



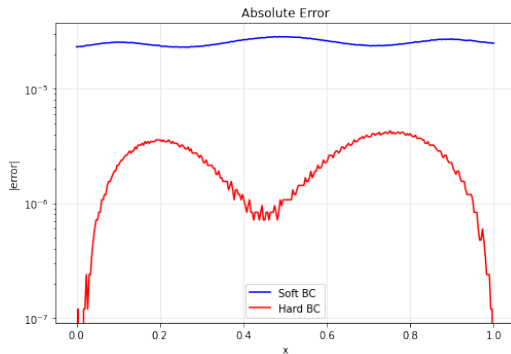
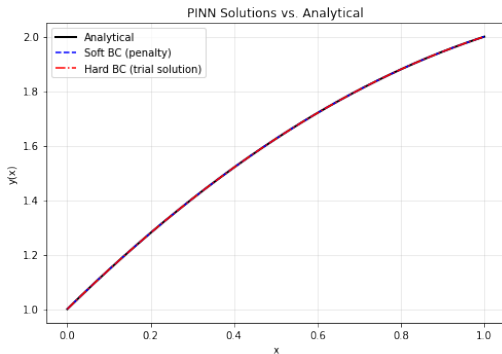
$B \cdot \mathcal{N}_\theta$: zero on $\partial\Omega$



Exactly zero on boundary

Then $\hat{u}(x, y) = \underbrace{A(x, y)}_{\text{matches BCs}} + \underbrace{B(x, y) \cdot \mathcal{N}_\theta(x, y)}_{\text{zero on } \partial\Omega}$ satisfies Dirichlet BCs **exactly**.

Soft vs. Hard BCs: Comparison



Both fit the solution (left). The difference is the error (right, log scale): the soft-BC run sits near 10^{-3} **including at the endpoints**, while the hard-BC run falls to 10^{-7} at the boundary, exact there by construction.

Notebook: 02_ODE_PINN_SoftVsHardBCs

Finger Exercise: Hard Boundary Conditions

Exercise (5 min)

Consider the same ODE $u''(x) = -1$ on $[0, 1]$ with $u(0) = u(1) = 0$.

1. Construct a **hard-BC trial solution** of the form $\hat{u}(x) = B(x) \cdot \mathcal{N}(x; \theta)$, where $B(x)$ enforces both BCs exactly. (Hint: B must vanish at $x = 0$ and $x = 1$.)
2. Verify that $\hat{u}(0) = \hat{u}(1) = 0$ for *any* network output $\mathcal{N}$.
3. What is the advantage of hard BCs over soft BCs?

Solution: Hard Boundary Conditions

Answer

1. $\hat{u}(x) = x(1 - x) \cdot \mathcal{N}(x; \theta)$. Since $B(x) = x(1 - x)$ vanishes at both endpoints, the BCs are satisfied exactly.
2. $\hat{u}(0) = 0 \cdot \mathcal{N}(0; \theta) = 0$. $\hat{u}(1) = 0 \cdot \mathcal{N}(1; \theta) = 0$. ✓
This holds regardless of what $\mathcal{N}$ outputs.
3. Hard BCs **eliminate the boundary loss term** entirely, the optimizer only needs to minimize the PDE residual. This reduces the number of hyperparameters (λ is gone) and often speeds up convergence.

Extension to 2D: Poisson Equation

Problem: $\Delta u = f(x, y)$ on $\Omega = (0, 1)^2$ with Dirichlet BCs.

Manufactured solution: $u^*(x, y) = x^2 + y + \sin(\pi x) \sin(\pi y)$.

Hard BCs via transfinite (Coons-patch) interpolation, the standard 4-side construction that matches each boundary function exactly along its edge:

$$\hat{u}(x, y) = A(x, y) + \underbrace{x(1-x)y(1-y)}_{B(x,y)} \cdot \mathcal{N}_\theta(x, y)$$

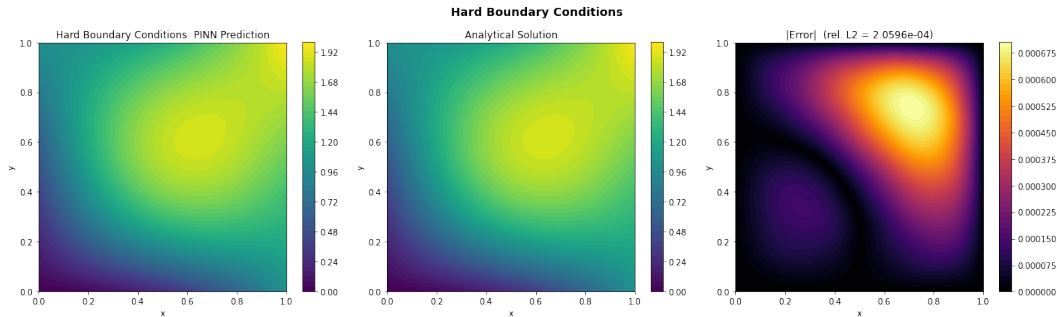
where $A(x, y)$ interpolates boundary data from all four sides:

$$A = (1-x)g_L(y) + xg_R(y) + (1-y)g_B(x) + yg_T(x) - \text{corners}$$

Laplacian via autograd:

$$\Delta \hat{u} = \frac{\partial^2 \hat{u}}{\partial x^2} + \frac{\partial^2 \hat{u}}{\partial y^2} \quad (\text{two chains of autograd.grad})$$

2D Poisson: Results



Hard-BC PINN, analytical solution, and absolute error. Relative L^2 error 2.06×10^{-4} against 1.36×10^{-3} for the tuned soft-penalty run: a **6.6×** improvement, at one fewer hyperparameter.

Notebook: 03_PDE_PINN_Poisson2D

Soft vs. Hard BCs: When to Use Which?

	Soft BCs	Hard BCs
Best for	Irregular domains, Robin or Neumann BCs	Simple domains, Dirichlet BCs, high accuracy
Pros	Flexible, easy to implement	Exact BCs, fewer hyperparams, better accuracy
Cons	λ tuning, approximate BCs	Needs analytical A , B ; harder for irregular domains

Practical recommendation

Start with **hard BCs** when the domain and boundary data allow it. Fall back to **soft BCs** for complex geometries or mixed boundary types.

Part IV

Economic Applications: The HJB Equation

The Hamilton–Jacobi–Bellman Equation

Generic continuous-time optimization:

$$\rho V(\mathbf{x}) = \max_c \{u(c) + V_{\mathbf{x}} \cdot f(\mathbf{x}, c)\}$$

- ▶ $V(\mathbf{x})$: **value function** (unknown)
- ▶ ρ : discount rate
- ▶ $u(c)$: flow utility
- ▶ $f(\mathbf{x}, c)$: law of motion for state $\mathbf{x}$
- ▶ $V_{\mathbf{x}} = \nabla V$: gradient of the value function

This is a PDE! First-order (or second-order with diffusion).

⇒ **Perfect fit for PINNs**: the network approximates V , autograd gives $V_{\mathbf{x}}$, and the HJB residual is the loss.

Cake-Eating Problem: Setup

A household with wealth $a(t)$ chooses consumption $c(t)$ to maximize lifetime utility:

$$\max_{\{c(t)\}} \int_0^{\infty} e^{-\rho t} \frac{c(t)^{1-\gamma}}{1-\gamma} dt \quad \text{s.t.} \quad \dot{a} = r a - c$$

HJB equation: $\rho V(a) = \max_c \left\{ \frac{c^{1-\gamma}}{1-\gamma} + V'(a)(r a - c) \right\}$

FOC: $c^{-\gamma} = V'(a) \Rightarrow c^* = (V'(a))^{-1/\gamma}$.

Analytical solution (with $\kappa = \frac{\rho - (1-\gamma)r}{\gamma} > 0$):

$$V^*(a) = \frac{\kappa^{-\gamma}}{1-\gamma} a^{1-\gamma}, \quad c^*(a) = \kappa a$$

Parameters: $\gamma = 2$, $\rho = 0.05$, $r = 0.03 \Rightarrow \kappa = 0.04$.

PINN for the HJB Equation: Hard-BC Trial Solution

Anchor + mask + scaled MLP. Let $x(a) = (a - a_{\min}) / (a_{\max} - a_{\min}) \in [0, 1]$ and S a value-scale constant:

$$\hat{V}(a) = V^*(a_{\min}) + x(a)[V^*(a_{\max}) - V^*(a_{\min})] + x(a)(1 - x(a)) S f_{\theta}(x(a)).$$

Endpoints are exact *by construction*, so the boundary loss is identically zero and the optimizer focuses on the HJB residual only.

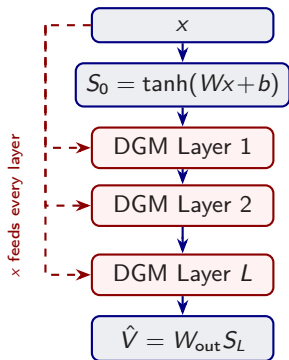
Consumption via FOC, $\hat{c}(a) = (\hat{V}'(a))^{-1/\gamma}$, with softplus safeguard on $\hat{V}'$.

Loss: only the interior HJB residual remains,

$$\ell = \frac{1}{N} \sum_i \mathcal{R}(a_i)^2, \quad \mathcal{R}(a) = \rho \hat{V} - \left[\frac{\hat{c}^{1-\gamma}}{1-\gamma} + \hat{V}'(ra - \hat{c}) \right].$$

Pedagogical point: a small MLP plus hard BCs beats a large gated net with a soft-BC penalty, because the boundary term no longer dominates the objective.

Optional Architecture for High-D PDEs: DGM



Deep Galerkin Method

(Sirignano & Spiliopoulos, 2018)

- Pays off only in **higher dimensions**, or where the PDE has sharp features.
- The input x enters **every** layer, not just the first — that is what resolves steep gradients.
- Each layer is **gated**, LSTM-style; the x -skips recall Highway Networks.

Not the default choice

For the 1D cake problem, the trial-solution MLP of the previous slide is simpler *and* more accurate.

Inside a DGM Layer

$S_\ell \in \mathbb{R}^U$ is the depth- ℓ hidden state, initialised as $S_0 = \tanh(Wx + b)$; $\odot$ is the Hadamard product; and Z, G, R are sigmoid gates valued in $[0, 1]$:

$$Z_\ell = \sigma(U^z x + W^z S_\ell + b^z)$$

$$G_\ell = \sigma(U^g x + W^g S_\ell + b^g)$$

$$R_\ell = \sigma(U^r x + W^r S_\ell + b^r)$$

$$H_\ell = \tanh(U^h x + W^h (S_\ell \odot R_\ell) + b^h)$$

$$S_{\ell+1} = (1 - G_\ell) \odot H_\ell + Z_\ell \odot S_\ell$$

Reading the gates

R_ℓ decides how much of the current state enters the candidate H_ℓ .

G_ℓ and Z_ℓ then blend that candidate against the state being carried forward.

Every one of them sees x directly.

Cost: four weight matrices per layer instead of one, so roughly $4\times$ the parameters of a plain MLP of the same width.

DGM Implementation (PyTorch)

```
class DGMNet(nn.Module):
    def __init__(self, dim, layers=4, units=64):
        super().__init__()
        self.input = nn.Linear(dim, units)
        self.z = nn.ModuleList(nn.Linear(dim, units) for _ in range(layers))
        self.wz = nn.ModuleList(nn.Linear(units, units) for _ in range(layers))
        # ... similar for g, r, h gates
        self.out = nn.Linear(units, 1)

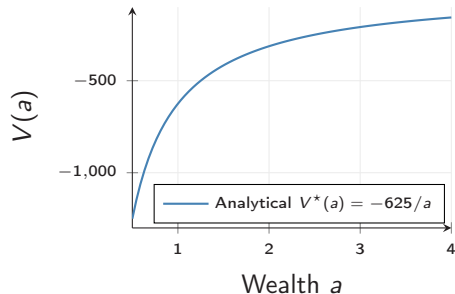
    def forward(self, x):
        s = torch.tanh(self.input(x))
        for l in range(len(self.z)):
            z = torch.sigmoid(self.z[l](x) + self.wz[l](s))
            g = torch.sigmoid(self.g[l](x) + self.wg[l](s))
            r = torch.sigmoid(self.r[l](x) + self.wr[l](s))
            h = torch.tanh(self.h[l](x) + self.wh[l](s * r))
            s = (1 - g) * h + z * s
        return self.out(s)
```

HJB Residual Implementation

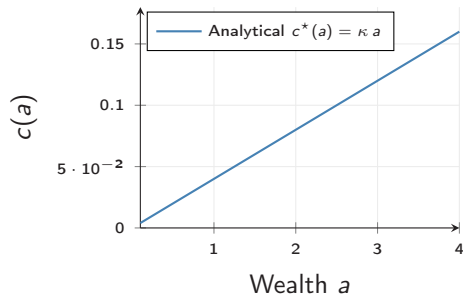
```
def pde_residual(model, a):  
    a.requires_grad_(True)  
    V = model(a) # value function  
    V_a = torch.autograd.grad( #  $V'(a)$  via autograd  
        V.sum(), a, create_graph=True)[0].squeeze(-1)  
  
    safe_Va = F.softplus(V_a) + 1e-6 # positivity guard  
    c = safe_Va.pow(-1.0 / gamma) # FOC: optimal c  
    u_c = c.pow(1-gamma) / (1-gamma) # flow utility  
  
    R = rho*V - (u_c + safe_Va*(r*a.squeeze(-1) - c))  
    return R # HJB residual
```

The softplus safeguard prevents $V'(a)^{-1/\gamma}$ from blowing up if $V'(a) \leq 0$ during early training.

Cake-Eating: the Closed-Form Benchmark



Value function $V(a)$ (closed form)



Consumption policy $c(a)$ (closed form)

These are the *exact* solutions, the target the PINN is scored against. Classroom run: HJB loss $\approx 3 \times 10^{-4}$, consumption relative L^2 error $\approx 6 \times 10^{-3}$. The notebook overlays the fitted $\hat{V}$ and $\hat{c}$.

Notebook: 04_Cake_Eating_HJB_PINN

Extensions: From Cake-Eating to Frontier Models

The PINN/DGM framework scales naturally:

1. **Income risk:** add diffusion $da = (ra - c + y)dt + \sigma dW \Rightarrow$ second-order HJB
2. **Aiyagari-type models:** HJB + Kolmogorov Forward Equation $\Rightarrow$ coupled PDE system
3. **General equilibrium:** prices clear markets $\Rightarrow$ hybrid DEQN + PINN
4. **High dimensions:** multiple assets, heterogeneous agents $\Rightarrow$ no grid needed!

See: Achdou et al. (2022), Fernandez-Villaverde et al. (2024), Gopalakrishna (2024).

Part VI

When to Use One, and What Goes Wrong

Should You Use a PINN at All?

On a 1D problem, finite differences win. Outright.

Notebook 07 solves the same partial-equilibrium HJB both ways:

- ▶ **Upwind implicit FD**: 13 iterations on a 500-point grid, then the error is whatever the discretization gives you — and you can shrink it by refining.
- ▶ **PINN**: thousands of epochs to reach 10^{-3} – 10^{-2} , with no comparable knob.

Anyone selling you PINNs as a universal replacement for a grid solver is overselling.

Use a PINN when the state space is high-dimensional, the domain is irregular, there are many discrete states, or you need the solution *and* its derivatives as a differentiable object. Otherwise, use the grid.

Where the trade flips

A tensor grid needs n^d points. At $n = 50$:

$d = 1$ 50

$d = 2$ 2,500

$d = 4$ 6.3 million

$d = 6$ 1.6×10^{10}

Collocation cost is set by the *sample size you choose*, not by d .

The Other Half: Inverse Problems

So far the PDE was known and we solved it. Raissi et al. (2019) also treat the reverse.

Forward problem

Parameters μ given, solve for u .

$$\ell(\theta) = \|\mathcal{D}_\mu[\mathcal{N}_\theta]\|^2 + \text{BCs}$$

Inverse problem

Some data on u observed, solve for u and μ .

$$\ell(\theta, \mu) = \|\mathcal{D}_\mu[\mathcal{N}_\theta]\|^2 + \|\mathcal{N}_\theta - u^{\text{obs}}\|^2$$

- ▶ μ is just **another set of trainable parameters**: one extra tensor, same optimizer, same loop.
- ▶ In economics μ is a **structural parameter** — risk aversion, a discount rate, an adjustment cost — and the data term is a moment condition.
- ▶ This is estimation and solution in **one** optimization, rather than a solver nested inside an estimation loop.

Connects directly to the deep-surrogate approach to structural estimation from Day 1.

When PINNs Fail: the Multi-Component Loss, Again

The soft-BC loss is a weighted sum of residuals that live at different scales:

$$\ell_{\theta} = \underbrace{\lambda_{\text{pde}} \ell_{\text{pde}}}_{\text{interior}} + \underbrace{\lambda_{\text{bc}} \ell_{\text{bc}}}_{\text{boundary}} + \underbrace{\lambda_{\text{ic}} \ell_{\text{ic}}}_{\text{initial}}$$

This is exactly the Day 1 pathology.

- ▶ A residual carrying $S^2 \partial_{SS} V$ is not in the same units as one carrying V itself.
- ▶ The largest-scale component captures the gradient; the others are effectively ignored.
- ▶ Adaptive weights cannot close a gap of several orders of magnitude.

On Day 1 this bought $\approx 110\times$ over equal weighting, where the best adaptive scheme managed $2\times$ (02_DeepEquilibriumNets.pdf, Part IV). See Wang, Teng & Perdikaris (2021).

What to do, in order

1. **Non-dimensionalize** first — rescale so every residual is $O(1)$.
2. Enforce **structurally** where you can: a hard-BC trial solution deletes ℓ_{bc} from the sum.
3. Only then reach for adaptive weights.

Hands-On: What to Run

All seven notebooks are PyTorch, CPU-only, and self-contained.

Run them on [nuvolos.cloud](#) — no local setup needed.

Start here, in this order:

- ▶ The simplest possible PINN: $y'' = -1$ with zero BCs 01
- ▶ Soft penalty vs. hard trial solution — the core choice 02
- ▶ The first economic application: the cake-eating HJB 04

Then, at your own pace:

- ▶ 2D Poisson: hard BCs by transfinite interpolation 03
- ▶ Black–Scholes, Greeks for free by autodiff 05
- ▶ PE HJB with discrete income, against an FD benchmark 07

Build one yourself (~30 min)

Notebook 06 is a fill-in-the-blank exercise: a PINN for $u'' + u = 0$ on $[0, \pi]$, solution $\sin(x)$. Solutions follow each task in the same file.

Summary and Key Takeaways

Method

PINNs embed **differential equations** directly into the neural network loss function via **automatic differentiation**.

Boundary Conditions

Hard BCs (trial solutions) eliminate tuning and ensure exact satisfaction. Use **soft BCs** for complex domains.

Applications

HJB equations, Black–Scholes, and any continuous-time model. Scales to **high dimensions** without grids.

From DEQNs to PINNs:

- ▶ **Same philosophy:** economic/financial equations *are* the loss
- ▶ **New ingredient:** `torch.autograd.grad` for derivatives of $\mathcal{N}_\theta$
- ▶ **Broader scope:** any PDE, not just algebraic equilibrium conditions

Questions?

Simon Scheidegger

HEC, University of Lausanne

<https://sischei.github.io>

Materials: github.com/sischei/deep_learning_padova_0926 Deep Learning for
Continuous-Time Models

Companion Notebooks:

- 01 ODE PINN (zero BCs)
- 02 Soft vs. hard BCs
- 03 2D Poisson
- 04 Cake-eating HJB
- 05 Black-Scholes
- 06 Exercise: build a PINN
- 07 PE HJB: FD vs. PINN

Next up:

11:00–12:30 · Yucheng Yang